KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

School of Computer Engineering

Digital Image Watermarking and Steganography

Embedding, Extraction and Robustness Analysis in Python

Subject: Image Processing and Applications (CS-30015)

Autumn Semester 2026

Submitted by:

Sayangsu Dawn (24155052)

CSE-33

Under the guidance of: Dr. Sricheta Parui

Abstract

This project studies how information can be hidden inside a digital image and how well it survives. Two classical approaches are implemented from scratch in NumPy: least-significant-bit (LSB) steganography in the spatial domain, and a blind watermark in the 8x8 block DCT domain based on quantization index modulation (QIM), with an optional repetition code and a secret key that selects the embedding positions. Both are evaluated with the same images, the same payload and the same metrics: PSNR, SSIM, bit error rate (BER) and a chi-square steganalysis test.

LSB embedding is almost invisible (PSNR 51.16 dB when the whole image carries data at 1 bit per pixel) and gives 8192 bytes of capacity in a 256x256 image, but it does not survive JPEG compression: the bit error rate is 0.457 even at quality 90, which is no better than guessing. The DCT-QIM watermark with step size 24 and 4-fold repetition carries a 256-bit logo at 44.85 dB PSNR and is recovered without error after JPEG quality 50, Gaussian noise of standard deviation 5 and a brightness shift. Neither method survives blurring or rescaling. A chi-square test flags LSB embedding at every rate tested from 10% upward, but does not flag the DCT watermark.

The report also shows the trade-off between invisibility and robustness as the QIM step size changes, explains why repetition coding helps against random noise but not against compression, and lists the novelty, limitations and future scope of the project.

Keywords: digital watermarking, steganography, LSB, DCT, quantization index modulation, robustness, JPEG, steganalysis, chi-square test, PSNR, SSIM.

Table of Contents

Abstract **2**

1. Introduction **4**

1.4 Novelty and contribution 4

2. Proposed Methodology **5**

2.1 Data used and its source 5

3. Experiment Segment **6**

Part 1: LSB capacity and imperceptibility 6

Part 2: DCT-domain watermark 7

Part 3: Robustness to attacks 8

Part 4: Step size and repetition 9

Part 5: Steganalysis 10

Part 6: Colour image 11

Part 7: Discussion 11

4. Literature Referred or Reviewed **12**

5. Comparison with Others **13**

6. Conclusion **13**

7. Future Work (Future Scope) **14**

8. GitHub Link **14**

9. Bibliography **14**

10. References **15**

1. Introduction

Sending a secret message, or proving that an image belongs to its owner, are two old problems with a common technical core: data has to be placed inside an image so that people looking at the image do not notice it. Steganography hides the existence of a message; watermarking hides a mark that must stay readable even if the image is later compressed, filtered or edited. The same pixel can serve either purpose, but the requirements pull in opposite directions: the more data is hidden, and the more strongly, the more visible the change and the easier it is to detect.

This project builds the two basic techniques and measures that conflict. It connects topics from the course: bit-plane analysis and spatial-domain processing (LSB), the discrete cosine transform and quantization (the same machinery as JPEG), noise models, filtering, histogram analysis and image quality metrics.

1.1 Objectives

- Implement LSB embedding and a DCT-domain QIM watermark from scratch, without watermarking libraries.
- Measure invisibility with PSNR and SSIM, and capacity in bits per pixel.
- Measure robustness as the bit error rate after 17 different attacks.
- Study the effect of the QIM step size and of repetition coding.
- Test whether LSB embedding can be detected with a chi-square steganalysis test.

1.2 Tools and test data

The code is written in Python with NumPy [14], SciPy, scikit-image [13], Matplotlib and Pillow. The test images are the standard grayscale Cameraman image and the colour Astronaut image, both resized to 256x256. A single command (python main.py) runs every experiment with fixed random seeds and writes all figures and tables to a results folder, so every number in this report can be reproduced.

1.3 Organisation of the report

Section 1.4 states the novelty and contribution. Section 2 describes the methodology and the data. Section 3 presents the experiments in six parts followed by a discussion. Section 4 reviews the literature. Section 5 compares the methods with each other and with the figures reported in the literature. Sections 6 and 7 give the conclusion and future scope, Section 8 the GitHub link, and the report ends with the bibliography and references.

1.4 Novelty and contribution

LSB embedding, the DCT and QIM are all classical, and this project does not claim a new algorithm. Its contribution is a careful, reproducible comparison in which the two methods are treated identically. The specific contributions are:

- **One benchmark, two methods.** Both methods carry the same 256-bit payload, use the same secret-key mechanism for choosing positions, and are attacked by the same 17 attacks, so the bit error rates in one table can be compared directly.
- **A measured trade-off.** The step-size sweep shows where the watermark becomes robust: the bit error rate after JPEG quality 30 drops from 0.195 at step 24 to 0.000 at step 32, at a cost of 2.38 dB of PSNR.
- **An explanation, not just a table.** Repetition coding lowers the error under random noise (Gaussian noise of standard deviation 10: 0.215 to 0.059) but leaves the error after blur and JPEG quality 30 near total loss (about 0.19). The report explains this: those attacks move every copy of a bit in the same direction, so majority voting cannot correct them.

- **A caution about reading BER.** A bit error rate of about 0.19 for the DCT watermark is shown to mean complete loss, because the extractor then outputs all zeros and the logo contains 19.5% ones. Reading the chart against the usual 0.5 'guessing' line would hide this.
- **Visual and statistical invisibility.** A chi-square test is applied to both methods, so invisibility to the eye and invisibility to a detector are reported side by side.

2. Proposed Methodology

The pipeline has five stages. A payload is turned into a bit sequence; it is embedded in a cover image by one of the two methods; the stego or watermarked image is attacked; the payload is extracted; and the result is measured. Quality of the image is measured against the original cover, and quality of the extraction against the original payload.

2.1 Data used and its source

No external dataset was downloaded and no data was collected. The two test images are the standard Cameraman (grayscale) and Astronaut (colour) images distributed with scikit-image [13] (skimage.data.camera and skimage.data.astronaut). The Astronaut image is a NASA photograph and is in the public domain. They are stored in the images folder as camera.png and astronaut.png and resized to 256x256 pixels. Everything else is generated by the code:

- the payload: a 16x16 binary logo showing the initials 'SD' (256 bits), a short text message (264 bits), and random bits for the capacity test;
- the attacks: JPEG compression by Pillow, Gaussian noise, salt-and-pepper noise, Gaussian blur, a 3x3 median filter, cropping, rescaling and a brightness shift, all with fixed random seeds.

2.2 LSB steganography (spatial domain)

With k = 1 each payload bit replaces the least significant bit of one pixel; with k bits per pixel the k lowest bit planes are replaced. A pixel therefore changes by at most 2^k - 1 grey levels. Positions are either raster order or a pseudo-random permutation seeded with a secret key, so that extraction with a wrong key returns noise (Section 3, Part 1).

$$p' = (p \text{ AND } (255 - (2^k - 1))) \text{ OR } m, \quad 0 <= m < 2^k$$

2.3 DCT-domain watermark with QIM

The cover is divided into 8x8 blocks and each block is transformed with the orthonormal 2-D DCT. One mid-frequency coefficient, at position (3, 2), is chosen: low frequencies are visible if changed and high frequencies are removed by compression. The coefficient c is quantized with step size Δ, and the parity of the quantization index carries one bit:

$$c' = \Delta \cdot q, \quad q = \text{nearest integer to } c/\Delta \text{ with } q \bmod 2 = bit$$

Extraction is blind: the original image is not needed. The decoder transforms each block, rounds c/Δ and reads the parity. With a repetition factor r each bit is placed in r different blocks and decided by majority vote. A 256x256 image has 1,024 blocks, so 256 bits with r = 4 uses every block.

2.4 Attacks

Seventeen attacks plus the no-attack case are applied one at a time: JPEG at quality 90, 70, 50 and 30; Gaussian noise with standard deviation 2, 5 and 10; salt-and-pepper noise at 1% and 5%; Gaussian blur with σ = 0.8 and 1.5; a 3x3 median filter; zeroing the bottom 10% and 25% of the rows (cropping); downscaling to 75% and 50% and back; and a brightness increase of 20.

2.5 Steganalysis

The chi-square test of Westfeld and Pfitzmann [4] looks at pairs of grey values (2i, 2i+1). In a natural image the two counts in a pair usually differ. Replacing least significant bits with random data pushes the counts in each pair towards equality. The statistic measures how close the observed counts are to equal counts, and a p-value close to 1 suggests that data is hidden. The test is run on the part of the image that was written to.

2.6 Evaluation metrics

$$PSNR = 10 \cdot \log10(255^2 / MSE) \qquad BER = (number\ of\ wrong\ bits) / (number\ of\ bits)$$

SSIM [5] compares local luminance, contrast and structure on a 0 to 1 scale. PSNR and SSIM are computed between the cover and the stego image. BER is computed between the embedded and the extracted bits. Normalised correlation (NC) is also implemented in the metrics module.

2.7 Implementation

The code is organised as one module per task (lsb, dct_wm, attacks, steganalysis, metrics), a runner main.py that executes all experiments, and 18 unit tests. The tests check, for example, that LSB extraction returns exactly what was embedded for k = 1 to 4, that no pixel changes by more than 2^k - 1, that extraction with a wrong key fails, that the DCT watermark survives JPEG better than LSB on a test image, and that the chi-square test responds to full-capacity embedding.

3. Experiment Segment

The experiments are presented in six parts, followed by a discussion (Part 7). All figures and tables were generated by the project code.

Part 1: LSB capacity and imperceptibility

Random payloads that fill the whole image were embedded with k = 1 to 4 bits per pixel (Table 1). Every payload was extracted exactly. Each additional bit plane roughly doubles the error energy, so PSNR falls by about 6 to 7 dB per plane, and from k = 3 the change becomes visible in smooth regions such as the sky, where banding appears; at k = 4 it is clearly noise (Figure 1).

Table 1: LSB embedding at full capacity in a 256x256 image.

k (bits/pixel)	Capacity (bytes)	PSNR (dB)	SSIM	Exact recovery
1	8,192	51.16	0.996	yes
2	16,384	44.14	0.980	yes
3	24,576	37.93	0.932	yes
4	32,768	31.88	0.805	yes

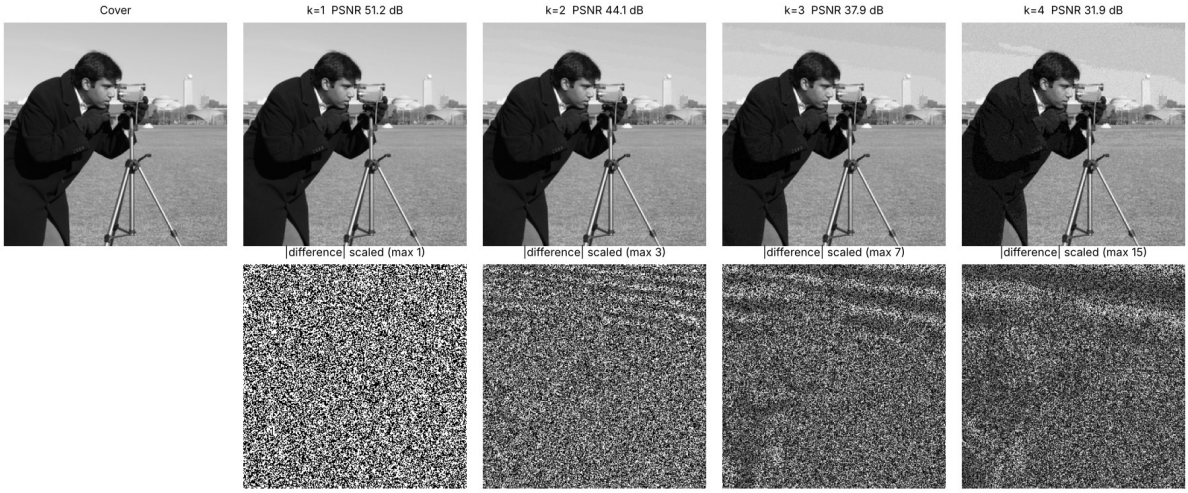


Figure 1: Stego images for k = 1 to 4 (top) and the scaled absolute difference from the cover (bottom).

A short text message ('Sayangsu Dawn | CSE-33 | 24155052', 264 bits) was hidden with the secret key and recovered exactly (PSNR 74.68 dB). Extraction with a key that differs by one returned unreadable bytes, which shows that the key, and not just the knowledge of the method, is needed to read the message. Figure 2 shows why LSB embedding is invisible: the least significant bit plane of a natural image already looks like noise, and so the hidden bits are not visible in it.

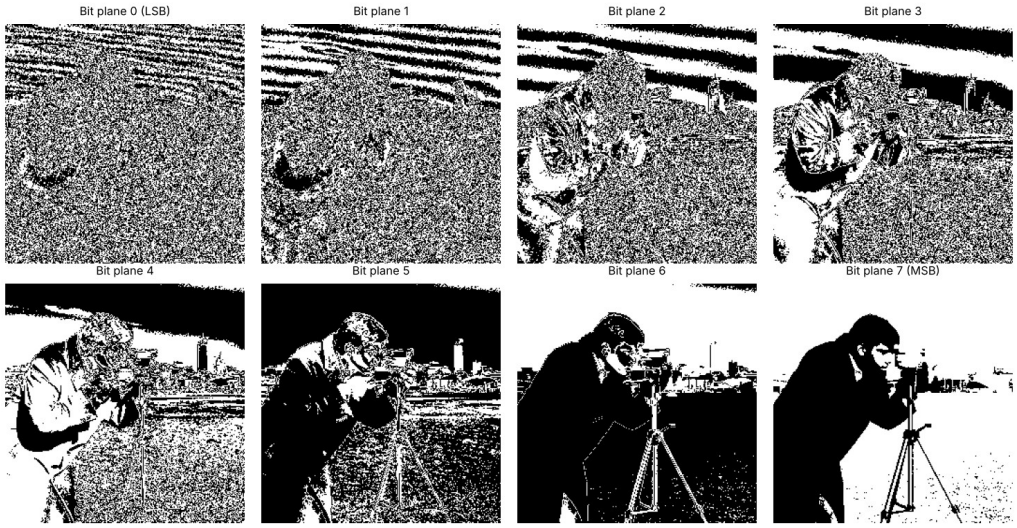


Figure 2: The eight bit planes of the Cameraman image.

Part 2: DCT-domain watermark

The 256-bit logo was embedded with Δ = 24 and r = 4. The result has PSNR 44.85 dB and SSIM 0.985. The change is a faint texture that is visible only when the difference is amplified (Figure 3). The logo was recovered with BER 0.000 without attack and 0.000 after JPEG quality 50.

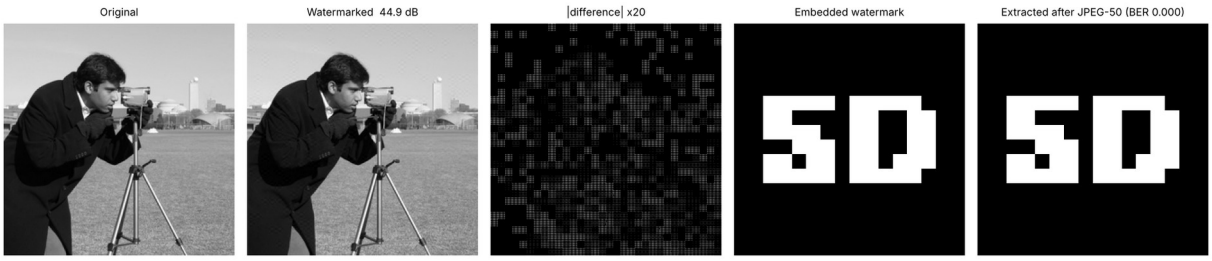


Figure 3: DCT watermark: original, watermarked image, amplified difference, embedded logo and logo extracted after JPEG quality 50.

Part 3: Robustness to attacks

Table 2 gives the bit error rate of the three configurations after each attack and Figure 4 plots the same data. All three recover the payload without error when nothing is done to the image. For the same 256-bit payload, LSB changes the image by 75.26 dB PSNR, DCT-QIM with r = 1 by 50.76 dB and DCT-QIM with r = 4 by 44.85 dB.

Table 2: Bit error rate after each attack (256-bit payload, Cameraman 256x256).

Attack	LSB (k=1)	DCT-QIM r=1	DCT-QIM r=4
No attack	0.000	0.000	0.000
JPEG Q90	0.457	0.000	0.000
JPEG Q70	0.520	0.000	0.000
JPEG Q50	0.449	0.004	0.000
JPEG Q30	0.562	0.172	0.195
Gaussian noise s=2	0.527	0.000	0.000
Gaussian noise s=5	0.520	0.008	0.000
Gaussian noise s=10	0.445	0.215	0.059
Salt&pepper 1%	0.000	0.195	0.043
Salt&pepper 5%	0.016	0.457	0.293
Gaussian blur s=0.8	0.453	0.262	0.195
Gaussian blur s=1.5	0.496	0.199	0.195
Median 3x3	0.391	0.242	0.191
Crop 10% rows	0.020	0.027	0.004
Crop 25% rows	0.055	0.051	0.051
Rescale 0.75	0.465	0.230	0.188
Rescale 0.5	0.516	0.211	0.195
Brightness +20	0.004	0.004	0.000

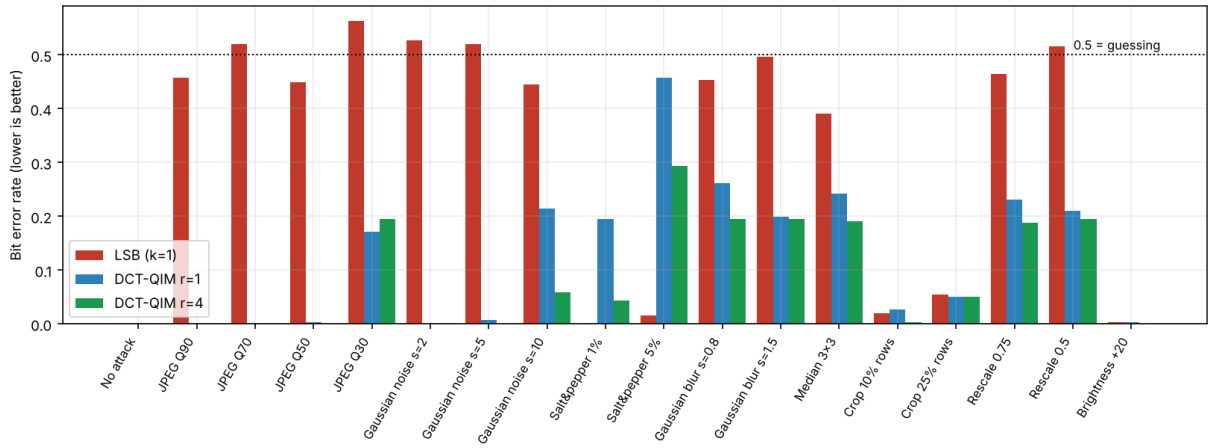


Figure 4: Bit error rate after each attack for the three configurations.

The results fall into three groups.

- **LSB.** LSB fails completely against anything that changes pixel values slightly everywhere: JPEG at every quality (BER 0.457 to 0.562), Gaussian noise, blur and rescaling. A change of one grey level is enough to flip a least significant bit.
- **LSB under local damage.** LSB is the more robust method against attacks that touch only some pixels. Only 25% of the pixels were cleared by the 25% crop, and those cleared pixels cause an error only where the hidden bit was a 1, which gives BER 0.055. Salt-and-pepper noise at 1% altered no bit that carried data (BER 0.000). This is partly luck: with only 256 of 65,536 pixels carrying data, about 3 hidden bits are expected to be hit at 1%.
- **DCT-QIM.** DCT-QIM is recovered without error after JPEG quality 90, 70 and 50 (r = 4), Gaussian noise of σ = 2 and 5, and a brightness change (a brightness shift only changes the DC coefficient, which is not used). It fails at JPEG quality 30 (BER 0.195), blur, median filtering and rescaling, which remove mid-frequency energy.

The DCT bit error rates of about 0.19 to 0.26 need careful reading. A low-pass attack shrinks the mid-frequency coefficient towards zero, so round(c/Δ) becomes 0, the parity is even, and every bit is read as 0. The error is then the fraction of ones in the logo, which is 0.195. A BER of 0.195 therefore means that the watermark is completely destroyed, even though it is far below 0.5.

Part 4: Step size and repetition

The step size Δ controls the trade-off (Table 3, Figure 5). Small steps keep the image almost unchanged but the watermark does not survive compression or noise: at Δ = 8 the BER after JPEG quality 50 is 0.242. From Δ = 16 the watermark survives JPEG quality 50 and noise of σ = 5, and from Δ = 32 it also survives JPEG quality 30, at 42.47 dB PSNR.

Table 3: Effect of the QIM step size (r = 4).

Step Δ	PSNR (dB)	SSIM	BER JPEG Q50	BER JPEG Q30	BER noise σ=5
8	53.23	0.998	0.242	0.195	0.254
12	50.10	0.995	0.195	0.215	0.066
16	47.77	0.992	0.004	0.195	0.008
24	44.85	0.985	0.000	0.195	0.000
32	42.47	0.974	0.000	0.000	0.000
48	39.32	0.950	0.000	0.000	0.000
64	37.14	0.925	0.000	0.000	0.000

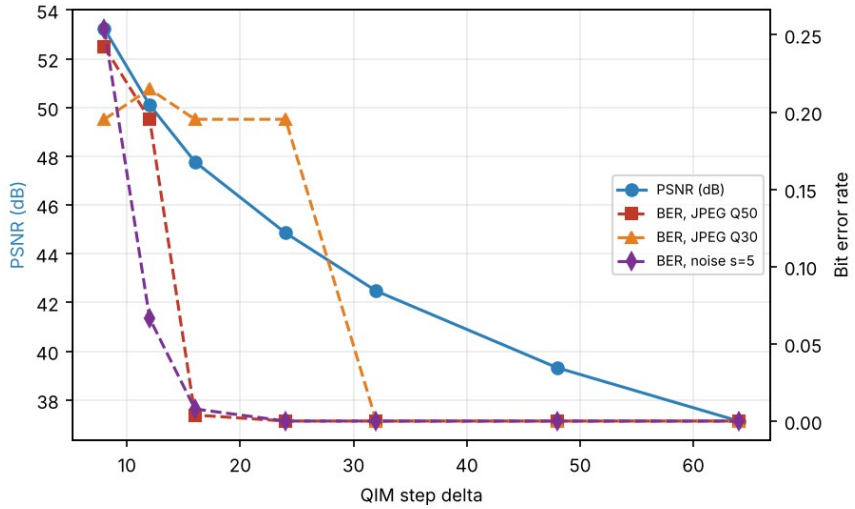


Figure 5: PSNR (left axis) and bit error rate (right axis) against the QIM step size.

Table 4: Effect of repetition coding (Δ = 24).

Repetition r	PSNR (dB)	BER JPEG Q50	BER JPEG Q30	BER crop 25%
1	50.76	0.004	0.172	0.051
2	47.84	0.000	0.195	0.078
4	44.85	0.000	0.195	0.051

Repetition does not help against JPEG quality 30 (BER 0.172 for r = 1 and 0.195 for r = 4), because compression removes the coefficient in every copy in the same way. It helps against noise that acts independently on each block (Table 2), and it costs 5.90 dB of PSNR because more blocks are modified. For compression, a larger step is the better tool.

Part 5: Steganalysis

Table 5: Chi-square test on LSB embedding (k = 1, raster order).

Embedding rate	PSNR (dB)	Chi-square p-value
0%	-	0.000
10%	61.16	0.994
25%	57.17	0.926
50%	54.13	1.000
100%	51.13	1.000

The clean image gives a p-value of 0.000, while LSB embedding gives 0.994 at a rate of 10% and at least 0.926 at every rate from 10% up. So the test detects LSB embedding reliably even though at full rate the image changes by only about 51 dB PSNR, which is invisible to the eye. The DCT-domain watermark was not flagged (p = 0.000). Embedding was in raster order here, so that the test window is known; with a secret key the positions are scattered and this simple test is expected to be less sensitive (Section 6, limitations).

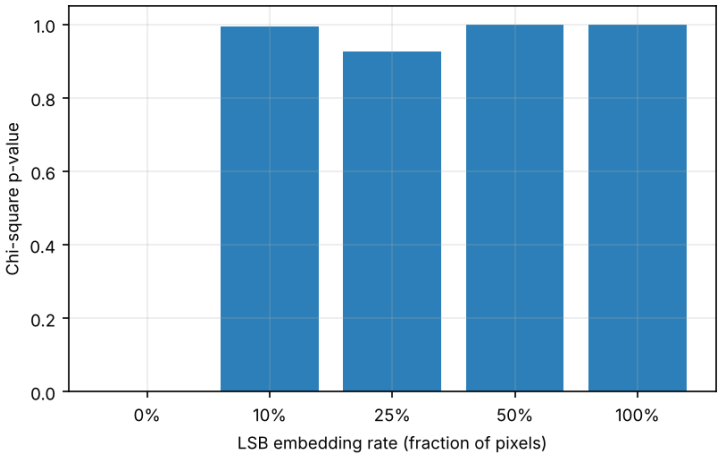


Figure 6: Chi-square p-value against LSB embedding rate (0% is the clean image).

Part 6: Colour image

For the colour Astronaut image the watermark was embedded in the luminance (Y) channel of the YCbCr representation (Δ = 24, r = 4), converted back to RGB and attacked with JPEG. The RGB image has PSNR 42.04 dB and SSIM 0.987. Table 6 shows the extraction result. Even without attack the BER is 0.016 (4 of 256 bits): converting Y to RGB and back rounds to integers and moves a few coefficients across a quantization boundary. The watermark survives JPEG down to quality 50 with at most 5 wrong bits and fails at quality 30.

Table 6: Watermark in the luminance channel of the colour image.

Attack	BER
No attack	0.016
JPEG Q90	0.020
JPEG Q70	0.020
JPEG Q50	0.000
JPEG Q30	0.188

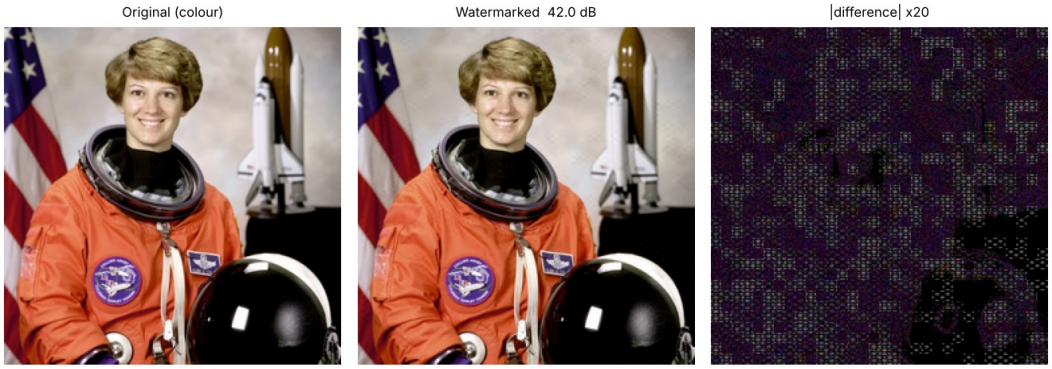


Figure 7: Colour image, watermarked image and amplified difference.

Part 7: Discussion

- A method that is invisible is not necessarily robust. LSB has the smallest visual change and the worst robustness to compression; DCT-QIM trades 25 to 30 dB of PSNR (for the same 256 bits) for survival of JPEG.

- Robustness is decided by which information the attack removes. JPEG and blur remove high and mid frequencies, so the watermark has to be placed where the attack leaves energy, or made strong enough to cross the quantization threshold.
- No single setting is best. Steganography wants the smallest change and tolerates a fragile message, whereas watermarking wants survival and tolerates a visible texture.
- Invisibility to the eye is different from invisibility to a detector: LSB at 51 dB PSNR is flagged by the chi-square test.

4. Literature Referred or Reviewed

This section summarises the literature that was read for each part of the project. The textbooks in the Bibliography were the main source for the course material.

4.1 Information hiding

Bender et al. [1] and Petitcolas, Anderson and Kuhn [2] give the framework in which steganography and watermarking are placed, with their different requirements of capacity, invisibility and robustness. Cheddad et al. [3] survey image steganography and organise the methods into spatial-domain, transform-domain and adaptive groups, which is the grouping used in this project.

4.2 Spatial-domain embedding and its detection

LSB replacement is the simplest spatial method. Westfeld and Pfitzmann [4] showed that it leaves a statistical trace in the histogram: the counts of the two values in each pair (2i, 2i+1) become equal. This is the chi-square test of Part 5. Fridrich, Goljan and Du [6] developed more sensitive detectors for LSB embedding in grayscale and colour images; these are left for future work.

4.3 Transform-domain watermarking

Cox et al. [7] introduced spread-spectrum watermarking, in which a pseudo-random sequence is added to the significant transform coefficients so that it survives common processing. Barni et al. [8] proposed a DCT-domain system for robust image watermarking. Chen and Wornell [9] introduced quantization index modulation, which hides bits in the choice of quantizer and, unlike additive schemes, is not disturbed by the image itself; this is the embedding rule used here. The DCT was introduced by Ahmed, Natarajan and Rao [10] and is the transform used in JPEG [11], which is why a DCT-domain mark interacts closely with JPEG compression.

4.4 Quality measures

PSNR is simple but only loosely related to perceived quality [12]. SSIM [5] compares local structure and was designed to follow human judgement more closely. Both are reported for invisibility, and BER for extraction.

Table 7: Summary of the literature reviewed and how it is used in this project.

Reference	Contribution	Used in this project for
Bender et al. [1]; Petitcolas et al. [2]	Information hiding framework	Problem set-up, requirements
Cheddad et al. [3]	Survey of image steganography	Choice of spatial and transform methods
Westfeld, Pfitzmann [4]	Chi-square attack on LSB	Steganalysis, Part 5
Wang et al. [5]	Structural similarity index	SSIM metric
Cox et al. [7]; Barni et al. [8]	Robust transform-domain watermarking	Motivation for the DCT approach
Chen, Wornell [9]	Quantization index modulation	Embedding rule, Section 2.3
Ahmed et al. [10]; Wallace [11]	DCT; JPEG standard	Block transform, JPEG attack

5. Comparison with Others

This section compares the methods of the project with each other, since all of them were measured on the same image, payload and attacks, and then with what the literature reports. No numbers from other papers are quoted, because their images, payloads and settings differ and the numbers would not be comparable.

5.1 LSB against DCT-QIM

Table 8: Side-by-side comparison of the three configurations.

Property	LSB (k=1)	DCT-QIM r=1	DCT-QIM r=4
Domain	Spatial	8x8 DCT	8x8 DCT
Payload used (bits)	256	256	256
Maximum capacity (256x256)	65,536 bits (k=1)	1,024 bits	256 bits
PSNR (dB)	75.26	50.76	44.85
SSIM	1.000	0.997	0.985
BER, JPEG Q50	0.449	0.004	0.000
BER, noise σ=5	0.520	0.008	0.000
BER, noise σ=10	0.445	0.215	0.059
BER, blur σ=1.5	0.496	0.199	0.195
BER, crop 25%	0.055	0.051	0.051
Detected by chi-square	Yes ($p \geq 0.92$)	No	No

LSB offers about 64 times the capacity of the DCT scheme with 1,024 blocks, and a change that is hard to see, but the payload is lost under JPEG, noise and blur. DCT-QIM carries less, changes the image more, and keeps the payload through JPEG quality 50 and moderate noise. The choice depends on whether the image will be processed after embedding.

5.2 Agreement with the literature

- The failure of LSB under lossy compression and the need for a transform domain for robust marking agree with the arguments in [1], [3] and [7].
- The finding that the LSB plane is detectable by the chi-square test at partial rates agrees with [4].
- The finding that QIM survives only while the added distortion stays below the quantizer step is the behaviour that [9] predicts: bits are lost when the attack moves a coefficient past half a step.
- The step size trade-off between PSNR and robustness (Table 3) is the usual trade-off of robust watermarking [7], [8].

6. Conclusion

Two embedding methods were built from scratch and compared under 17 attacks. LSB embedding is the less visible (75.26 dB for 256 bits; 51.16 dB at full capacity), has high capacity, and is destroyed by JPEG even at quality 90. DCT-QIM with $\Delta = 24$ and $r = 4$ was recovered exactly after JPEG quality 50, noise of $\sigma = 5$ and a brightness change at 44.85 dB, and a step of 32 extends this to JPEG quality 30. Neither method survived blur, median filtering or rescaling. Repetition coding helps only against independent noise. The chi-square test detected LSB embedding but not the DCT watermark.

Limitations:

- Only two test images were used, and attacks use one random seed, so no mean or standard deviation is reported.
- Each bit error rate comes from a 256-bit payload, so one bit changes the value by 0.004.
- The chi-square test was run on raster-order embedding. With a secret key the positions are scattered, and the simple test is expected to be weaker; stronger detectors were not implemented.
- Attacks that need synchronisation (rotation, translation, large crops) are not handled, and cropping was modelled as clearing rows.
- Watermark extraction was tested with a known step size Δ, a known coefficient position and a known key.
- The step size and repetition factor were chosen by trying values on the test image, not tuned on a separate set.

7. Future Work (Future Scope)

- **Better coding.** Replace the repetition code with an error-correcting code such as BCH, which can also correct the systematic errors that repetition cannot.
- **Other domains.** Embed in wavelet (DWT) or SVD coefficients and compare with the DCT scheme in the same benchmark.
- **Adaptive step size.** Make Δ depend on block content using a perceptual model so that textured blocks carry a larger step and flat blocks a smaller one.
- **Geometric robustness.** Add a synchronisation template so that the watermark can survive small rotations and rescaling.
- **Stronger steganalysis.** Implement the more sensitive detectors of [6] and test keyed embedding against them.
- **Larger test set.** Repeat the experiments on a larger set such as the Kodak images and report mean and standard deviation.

8. GitHub Link

The source code, scripts and results of this project are available on GitHub:

https://github.com/YOUR-USERNAME/IPA_Project

9. Bibliography

Books, standards and documentation that were studied but are not cited by number in the text:

1. R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. New York, NY, USA: Pearson Education, 2018.
2. A. K. Jain, Fundamentals of Digital Image Processing. Englewood Cliffs, NJ, USA: Prentice Hall, 1989.
3. I. J. Cox, M. L. Miller, J. A. Bloom, J. Fridrich, and T. Kalker, Digital Watermarking and Steganography, 2nd ed. Burlington, MA, USA: Morgan Kaufmann, 2008.
4. ITU-T Recommendation T.81, Information technology: Digital compression and coding of continuous-tone still images (JPEG), 1992.
5. scikit-image documentation, https://scikit-image.org/docs/stable/ (image processing functions and bundled test images).
6. NumPy, SciPy, Matplotlib and Pillow online documentation (array operations, DCT routines, plotting and the JPEG encoder used for the attack).

10. References

[1] W. Bender, D. Gruhl, N. Morimoto, and A. Lu, “Techniques for data hiding,” IBM Syst. J., vol. 35, no. 3–4, pp. 313–336, 1996.

[2] F. A. P. Petitcolas, R. J. Anderson, and M. G. Kuhn, “Information hiding—a survey,” Proc. IEEE, vol. 87, no. 7, pp. 1062–1078, 1999.

[3] A. Cheddad, J. Condell, K. Curran, and P. McKevitt, “Digital image steganography: Survey and analysis of current methods,” Signal Process., vol. 90, no. 3, pp. 727–752, 2010.

[4] A. Westfeld and A. Pfitzmann, “Attacks on steganographic systems,” in Information Hiding, Lecture Notes in Computer Science, vol. 1768. Berlin, Germany: Springer, 2000.

[5] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, 2004.

[6] J. Fridrich, M. Goljan, and R. Du, “Detecting LSB steganography in color and gray-scale images,” IEEE Multimedia, vol. 8, no. 4, pp. 22–28, 2001.

[7] I. J. Cox, J. Kilian, F. T. Leighton, and T. Shamoon, “Secure spread spectrum watermarking for multimedia,” IEEE Trans. Image Process., vol. 6, no. 12, pp. 1673–1687, 1997.

[8] M. Barni, F. Bartolini, V. Cappellini, and A. Piva, “A DCT-domain system for robust image watermarking,” Signal Process., vol. 66, no. 3, pp. 357–372, 1998.

[9] B. Chen and G. W. Wornell, “Quantization index modulation: A class of provably good methods for digital watermarking and information embedding,” IEEE Trans. Inf. Theory, vol. 47, no. 4, pp. 1423–1443, 2001.

[10] N. Ahmed, T. Natarajan, and K. R. Rao, “Discrete cosine transform,” IEEE Trans. Comput., vol. C-23, no. 1, pp. 90–93, 1974.

[11] G. K. Wallace, “The JPEG still picture compression standard,” Commun. ACM, vol. 34, no. 4, pp. 30–44, 1991.

[12] Q. Huynh-Thu and M. Ghanbari, “Scope of validity of PSNR in image/video quality assessment,” Electron. Lett., vol. 44, no. 13, pp. 800–801, 2008.

[13] S. van der Walt et al., “scikit-image: image processing in Python,” PeerJ, vol. 2, p. e453, 2014.

[14] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.

Thank You